

LangChain with Python

A comprehensive practical tutorial for modern LangChain applications, agents, tools, RAG, structured output, memory, evaluation, and production patterns

Updated for the modern LangChain Python API — August 2026

Table of Contents

1. What LangChain is and when to use it
2. Installation and project setup
3. Models and messages
4. Prompts and reusable prompt templates
5. Tool calling
6. Agents
7. Structured output
8. Retrieval and RAG
9. Documents, splitting, embeddings, and vector stores
10. Memory and conversation state
11. Streaming
12. Middleware, guardrails, retries, and human approval
13. LangGraph: when LangChain is not enough
14. LangSmith observability and evaluation
15. Testing and production architecture
16. Capstone: documentation assistant
17. Troubleshooting and common mistakes
18. Learning roadmap and cheat sheet

1. What LangChain is and when to use it

LangChain is an open-source framework for composing language models with tools, retrieval, structured output, agent runtimes, and middleware. The modern Python API emphasizes a standard model interface and the production-ready `create_agent` abstraction. LangChain agents are built on LangGraph underneath.

Think in layers: model → messages/prompts → tools → agent/workflow → retrieval → state/memory → observability/evaluation. You do not need every layer for every application. Use LangChain when the abstractions reduce complexity; for a tiny one-off model call, a provider SDK can be simpler.

2. Installation and project setup

Current LangChain Python documentation requires Python 3.10+. Provider integrations are separate packages. A typical OpenAI-based environment is:

```
python -m venv .venv
# macOS/Linux
source .venv/bin/activate
# Windows
# .venv\Scripts\activate

pip install -U langchain langchain-openai
pip install -U langchain-text-splitters langchain-community pypdf
pip install -U langsmith

# macOS/Linux
export OPENAI_API_KEY="your-key"
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="your-langsmith-key"
```

3. Models and messages

Chat models receive messages and return AI messages. Provider packages such as `langchain-openai` supply provider-specific implementations. Keep model configuration centralized; common controls include temperature, token limits, timeouts, and provider-specific parameters.

```
from langchain_openai import ChatOpenAI
from langchain.messages import SystemMessage, HumanMessage

model = ChatOpenAI(model="gpt-5.4", temperature=0, timeout=30)
messages = [
    SystemMessage(content="You are a concise Python tutor."),
    HumanMessage(content="Explain decorators."),
]
response = model.invoke(messages)
print(response.content)
```

4. Prompts and reusable prompt templates

Prompt templates separate application logic from wording. Make prompts explicit about the task, constraints, desired output, and what to do when evidence is insufficient.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a Python tutor. Explain clearly."),
    ("human", "Teach me about {topic}."),
])
messages = prompt.invoke({"topic": "context managers"})
response = model.invoke(messages)
print(response.content)
```

5. Tool calling

A tool is a callable function with a well-defined contract. The model decides when to call it; your application executes it and returns the result. Type annotations and docstrings help the model understand the tool. For network tools, validate inputs, use timeouts, handle failures, and never expose secrets.

```
from langchain.tools import tool

@tool
def add_numbers(a: float, b: float) -> float:
    """Add two numbers and return the result."""
    return a + b

print(add_numbers.invoke({"a": 2, "b": 3}))
```

6. Agents

An agent combines a model and tools and iterates through model calls and tool calls until it can answer or reaches a stop condition. The modern entry point is `create_agent`. Keep tools narrow, descriptions clear, permissions minimal, and iteration limits bounded. Prefer explicit workflows when the sequence is fixed.

```
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[add_numbers],
    system_prompt="You are helpful. Use tools when useful.",
)
result = agent.invoke({
    "messages": [{"role": "user", "content": "What is 17 + 25?"}]
})
print(result["messages"][-1].content)
```

7. Structured output

Free-form text is hard for software to consume reliably. Structured output lets your application receive typed data. Treat model output as untrusted input and validate it.

```
from pydantic import BaseModel, Field
from langchain.agents import create_agent

class Contact(BaseModel):
    name: str
    email: str
    confidence: float = Field(ge=0, le=1)

agent = create_agent(model="openai:gpt-5.4", response_format=Contact)
result = agent.invoke({"messages": [{
    "role": "user", "content": "Extract: Jane Doe, jane@example.com"
}]})
print(result["structured_response"])
```

8. Retrieval and RAG

Retrieval addresses finite context and private or changing knowledge. A RAG system retrieves relevant external knowledge at query time and supplies it to the model. 2-step RAG retrieves before generation and is predictable; agentic RAG gives an agent retrieval tools; hybrid RAG adds query rewriting, retrieval validation, or answer checks.

Conceptual pipeline: documents → chunks → embeddings → vector store → retriever → context → model → grounded answer.

```
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS

texts = [
    "LangChain provides abstractions for LLM applications.",
    "Retrieval fetches relevant external knowledge at query time.",
    "Agents can call tools in an iterative loop.",
]
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
db = FAISS.from_texts(texts, embeddings)
```

```

retriever = db.as_retriever(search_kwargs={"k": 3})
question = "What does retrieval do?"
docs = retriever.invoke(question)
context = "\n\n".join(d.page_content for d in docs)
prompt = f"""Answer using only the context below.
Context:
{context}

Question: {question}"""
answer = ChatOpenAI(model="gpt-5.4", temperature=0).invoke(prompt)
print(answer.content)

```

9. Documents, splitting, embeddings, and vector stores

LangChain represents source material as Document objects with page content and metadata. Split large documents into retrievable chunks. Start with moderate chunk size and overlap, then evaluate. Embeddings convert text into vectors; vector stores index those vectors for similarity search.

```

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = PyPDFLoader("data/manual.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=150
)
chunks = splitter.split_documents(docs)
print(len(docs), len(chunks))
print(chunks[0].metadata)

```

10. Memory and conversation state

Conversation state is information carried across agent invocations. In-memory checkpointing is useful for development; production applications should use persistent storage. Separate conversational summaries/preferences from durable business data, which should remain in your database.

```

from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent

checkpointer = InMemorySaver()
agent = create_agent(
    model="openai:gpt-5.4", tools=[], checkpointer=checkpointer
)
config = {"configurable": {"thread_id": "demo-user-1"}}
agent.invoke({"messages": [{"role": "user", "content": "My name is Alex."}]}, config)
result = agent.invoke({"messages": [{"role": "user", "content": "What is my name?"}]}, config)
print(result["messages"][-1].content)

```

11. Streaming

Streaming improves perceived latency by sending partial model or agent progress to a UI. LangChain/LangGraph support streaming execution and state; select a stream mode that matches your frontend.

```

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "Explain RAG."}]},
    stream_mode="updates",
):
    print(chunk)

```

12. Middleware, guardrails, retries, and human approval

Middleware can transform prompts, control tool selection, add retries/fallbacks, enforce rate limits, and implement guardrails. For sensitive actions such as payments, destructive writes, messages, or deployments, insert a human approval step.

Production safeguards include timeouts, bounded retries with backoff, idempotency, least-privilege tools, validation, and audit logs.

13. LangGraph: when LangChain is not enough

LangChain agents use LangGraph underneath. Use LangGraph directly when you need explicit state machines, durable execution, conditional routing, human interrupts, persistence, or highly customized orchestration.

A graph contains state, nodes that perform work, and edges that decide what happens next. If `create_agent` already solves the problem, a custom graph may add unnecessary complexity.

14. LangSmith observability and evaluation

As applications make multiple model and tool calls, tracing lets you inspect prompts, tool calls, intermediate results, latency, and failures. Evaluate final answers and intermediate behavior using normal, edge, adversarial, and regression datasets. Track correctness, groundedness, tool selection, latency, and cost.

```
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="your-key"
# Keep credentials in environment variables or a secret manager.
```

15. Testing and production architecture

Unit-test deterministic tools independently; mock external calls where appropriate. Integration-test real provider behavior with a smaller suite.

A practical architecture is: API/UI → application/service layer → LangChain agent or workflow → tools/retrievers → data/services. Keep business logic and secrets outside prompts.

```
# Production checklist
# - Timeouts on external calls
# - Bounded retries
# - Input/output validation
# - Least-privilege tools
# - Idempotent writes
# - Rate limiting
# - Persistent state when required
# - Tracing and alerts
# - Regression/evaluation suite
# - Cost and latency monitoring
```

16. Capstone: documentation assistant

Build a PDF documentation assistant using ingestion, chunking, embeddings, vector search, a retriever tool, an agent, and tracing. This is a realistic end-to-end project.

```
pip install -U langchain langchain-openai langchain-community \
    langchain-text-splitters langgraph langsmith pypdf faiss-cpu

from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

docs = PyPDFDirectoryLoader("data/").load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=150)
chunks = splitter.split_documents(docs)

from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.tools import tool

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
db = FAISS.from_documents(chunks, embeddings)
retriever = db.as_retriever(search_kwargs={"k": 4})

@tool
def search_docs(query: str) -> str:
    """Search the documentation and return relevant passages."""
    docs = retriever.invoke(query)
    return "\n\n".join(d.page_content for d in docs)

from langchain.agents import create_agent
```

```

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[search_docs],
    system_prompt=(
        "Use the documentation tool when the question requires "
        "information from the local corpus. If evidence is insufficient, say so."
    ),
)
result = agent.invoke({"messages": [{
    "role": "user", "content": "How do we configure authentication?"
}]})
print(result["messages"][-1].content)

```

17. Troubleshooting and common mistakes

Import errors usually mean a missing provider package, an outdated package, or an API change. Authentication errors usually mean a missing or invalid environment variable. Poor RAG answers often originate in chunking, retrieval, metadata, or source quality rather than the model.

If agents loop, improve tool descriptions, reduce ambiguity, set iteration limits, and validate tool outputs. If answers hallucinate, ground them in evidence, require an insufficient-evidence response, and evaluate against source documents.

18. Learning roadmap and cheat sheet

Recommended progression: model invocation → prompts → tools → agents → structured output → retrieval/RAG → state/memory → LangGraph → LangSmith/evaluation → production engineering.

```

# Model
model.invoke("question")

# Prompt
prompt = ChatPromptTemplate.from_messages([...])

# Tool
@tool
def my_tool(x: str) -> str: ...

# Agent
agent = create_agent(model, tools=[my_tool])

# Retrieval
docs = retriever.invoke("query")

# State
checkpointer = InMemorySaver()

```

References and further study

This tutorial is aligned with the current official LangChain Python documentation available in August 2026. LangChain evolves quickly, so verify provider-specific model names, package versions, and APIs against the official documentation before deployment.

Key official topics covered by the documentation include LangChain overview and installation, Quickstart, Agents, Tools, Retrieval/RAG, LangGraph agentic RAG, Middleware, and the Learn section.

Security note: never put API keys, credentials, private data, or production secrets directly in source code or prompts.